

Nomie AI Integration Engine - Architecture & Implementation Guide

Executive Summary

The Nomie AI Integration Engine is a sophisticated, modular system that enriches user tracking data with artificial intelligence capabilities. It provides Claude with comprehensive context about user activities, locations, goals, and social interactions, enabling intelligent analysis and personalized recommendations. This document serves as a technical reference for understanding the architecture and integrating new AI-powered features.

Table of Contents

- 1. [Architecture Overview](#)
- 2. [Core Components](#)
- 3. [Context System](#)
- 4. [AI Profiles](#)
- 5. [Data Flow](#)
- 6. [Integration Guide for New Components](#)
- 7. [Advanced Topics](#)

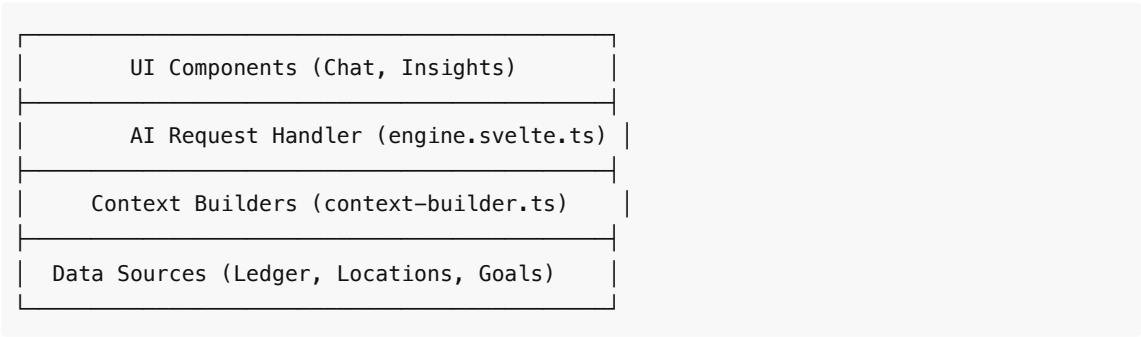
Architecture Overview

Design Philosophy

The AI integration engine follows these core principles:

- **Modular Design:** Each piece of functionality is independent and can be extended or replaced
- **Data Enrichment:** Raw tracking data is transformed into rich context that Claude can understand
- **Profile-Based Routing:** Different AI profiles specialize in different analysis types
- **Type Safety:** Full TypeScript typing ensures correctness across the system
- **Lazy Loading:** Context is built only when needed to minimize performance impact

System Layers



The system is organized in three main directories within the AI domain:

- `src/domains/ai/` - Core AI engine and profiles
- `src/domains/ai/profiles/` - AI profile definitions

- `src/domains/ai/context-builder.ts` - Context generation system
-

Core Components

1. AI Engine (`engine.svelte.ts`)

Purpose: Central orchestrator for all AI requests

Key Functions:

- `query()` - Single-shot AI requests with streaming support
- `streamQuery()` - Real-time streaming responses for chat
- `getDateRangeFromTimeframe()` - Converts timeframe IDs to date ranges

How It Works:

When a component requests AI assistance, the engine:

1. Receives an `AIRequest` with profile name, prompt, and optional context hints
2. Looks up the requested profile configuration
3. Calls the appropriate context builder based on profile type
4. Generates the system prompt with enriched context
5. Sends the request to Claude API
6. Returns the response (or streams it for real-time features)

Key Implementation Detail: The engine applies the "default timeframe" setting from preferences when no explicit date range is provided. This ensures consistent data windows across all AI features unless overridden by specific components.

2. Context Builder (`context-builder.ts`)

Purpose: Transforms raw user data into AI-readable context

Core Concept: The context builder implements a multi-profile context system where different AI use cases receive tailored data subsets.

Main Functions:

- `buildContext()` - Legacy context (metrics only, rarely used)
- `buildChatContext()` - Rich context with all data types (used by Chat and Insight)
- `buildSummaryContext()` - High-level overview (used for quick summaries)
- `buildMetricsContext()` - Metrics-focused (used for goal tracking)
- `buildJournalContext()` - Journal entries and reflection (used for narrative analysis)
- `buildPeopleContext()` - Social interaction analysis
- `buildLocationContext()` - Geographic patterns
- `buildCorrelationContext()` - Metric relationships and trends
- `buildTrendContext()` - Momentum and direction analysis
- `buildGoalProgressContext()` - Progress toward specific goals
- `buildHealthContext()` - Health metrics filtered for health-focused analysis
- `buildPatternContext()` - Temporal patterns (day-of-week, hour-of-day)

Data Extraction Flow:

Each context builder follows a consistent pattern:

1. Query logs from LedgerStore for the specified date range
2. Tokenize log notes to extract trackables, people, contexts, pointers
3. Aggregate data based on tracker math rules (sum, average, etc.)
4. Enrich with metadata (people counts, location names, goal progress)
5. Sort and rank by relevance (frequency, recency)
6. Return structured context object

3. Data Source Fetchers

Location Resolution:

- Fetches logs with GPS coordinates
- Resolves coordinates to saved location names using LocationStore
- Includes temporal information (last visit date, recent visit dates)
- Returns top 10 most-visited locations

Context & Pointer Extraction:

- Tokenizes note text looking for `+context` and `^pointer` patterns
- Groups by identifier and counts occurrences
- Tracks frequency and recent values
- Returns unique contexts and pointers found in logs

Person Interaction Tracking:

- Extracts `@person` mentions from notes
- Counts interactions per person
- Provides recent interaction values
- Enables social pattern analysis

Metric Aggregation:

- Extracts `#tracker` values from notes
 - Applies tracker-specific math rules
 - Groups by date for temporal analysis
 - Provides aggregated and recent values
-

Context System

Understanding Context Types

The context system provides specialized data views for different AI use cases:

RichContext

The most comprehensive context type, containing all available data:

- Complete metrics with temporal information
- All goals
- People interactions
- Locations with coordinates and visit dates
- Contexts and pointers
- Journal notes
- User summary

Used by: Chat, Insight profiles

SummaryContext

High-level overview for quick assessments:

- User summary only
- Top 3-5 goals
- No detailed data

Used by: Quick status checks, summary generation

MetricsContext

Focused on quantitative data:

- All metrics with aggregation info
- Goals (related to metrics)
- User summary

Used by: Goal tracking, Advice profile

JournalContext

Focused on qualitative data:

- Journal notes organized by date
- Related goals
- User summary

Used by: Narrative analysis, Journal reflection prompts

HealthContext

Filtered metrics for health analysis:

- Only health-related metrics (sleep, heart rate, exercise, mood, energy, stress)
- Goals filtered to health goals
- User summary

Used by: Health-focused analysis, wellness recommendations

LocationContext

Spatial and temporal patterns:

- Locations with visit history
- Day-of-week patterns
- Hour-of-day patterns
- User summary

Used by: Location-based insights, spatial behavior analysis

CorrelationContext

Relationship analysis:

- Derived correlations between metrics
- Trends and momentum
- Ranked by correlation strength
- Full rich context as foundation

Used by: Pattern discovery, behavioral insights

TrendContext

Temporal momentum analysis:

- Direction (up/down/flat) for each metric
- Momentum percentage change
- Last value and average comparison
- Ranked by significance

Used by: Trend forecasting, momentum-based recommendations

GoalProgressContext

Goal achievement tracking:

- Progress toward each goal (percentage complete)
- Current vs. target values
- Estimated days to completion
- Full rich context as foundation

Used by: Goal coaching, achievement predictions

PatternContext

Behavioral and temporal patterns:

- Day-of-week visit patterns
- Hour-of-day activity patterns
- Location patterns
- Frequency distributions

Used by: Habit formation, schedule optimization

Context Hints

Components can control context building through context hints:

```
// Pass specific date range
contextHints: {
  dateRange: {
    from: "2026-06-01",
    to: "2026-06-10"
  }
}

// Request specific metrics only
contextHints: {
  metrics: ["#sleep_aw", "#steps_aw", "#mood"]
}
```

If no hints are provided, sensible defaults apply:

- **Date Range:** Default timeframe from settings (typically last 30 days)
- **Metrics:** All tracked metrics in the period

AI Profiles

What is a Profile?

A profile is a specialized configuration that tells Claude how to behave for a specific use case. It defines:

- **System Prompt Template:** Instructions for Claude's role and behavior
- **Context Type:** Which context builder to use
- **Max Tokens:** Response length limit
- **Response Parser:** Function to parse Claude's response into a usable format
- **Temperature:** Creativity level (0.0 = deterministic, 1.0 = creative)

Current Profiles

Chat Profile (`src/domains/ai/profiles/chat.ts`)

- **Purpose:** Conversational analysis of tracking data
- **Context:** RichContext (complete data)
- **Tone:** Friendly, encouraging, analytical
- **Features:**
 - Understands location coordinates and can interpret them geographically
 - References specific metrics, patterns, and journal entries
 - Makes location-aware correlations
 - Provides detailed multi-paragraph responses
- **Temperature:** 0.7 (balanced creativity and accuracy)
- **Max Tokens:** 1000

Insight Profile (`src/domains/ai/profiles/insight.ts`)

- **Purpose:** Automated insight generation for dashboard widget
- **Context:** RichContext (complete data for deep analysis)
- **Tone:** Data-driven, specific, actionable
- **Features:**
 - Identifies trends and correlations
 - References actual metric values and social interactions
 - Considers location context and temporal patterns
 - Focuses on patterns not obvious from raw data
- **Temperature:** 0.7
- **Max Tokens:** 1000

Advice Profile (`src/domains/ai/profiles/advice.ts`)

- **Purpose:** Goal-focused recommendations
- **Context:** MetricsContext (metrics-focused)
- **Tone:** Coaching, goal-oriented, practical
- **Features:**
 - At least 2-3 recommendations must directly address goal achievement
 - Analyzes progress toward specific goals
 - Provides actionable, measurable advice
 - References actual metric values
 - Suggests behavioral context changes when appropriate
- **Response Format:** JSON array of advice items with priority levels
- **Temperature:** 0.4 (focused, deterministic)

- **Max Tokens:** 1200

Journal Profile (`src/domains/ai/profiles/journal.ts`)

- **Purpose:** Reflection and journaling prompts
- **Context:** JournalContext + LocationContext (qualitative + spatial)
- **Tone:** Thoughtful, introspective, personalized
- **Features:**
 - Generates 1 opening question + 2-3 follow-up questions
 - Tailored to user's specific data and locations
 - References social interactions and contextual environments
 - Asks about goal progress and experiences
 - Includes mood scale label for emotional reflection
- **Response Format:** JSON object with opening_question, follow_up_questions, mood_scale_label
- **Temperature:** 0.5 (balanced reflective tone)
- **Max Tokens:** 800

Data Profile (`src/domains/ai/profiles/data.ts`)

- **Purpose:** Data summarization and interpretation
- **Context:** MetricsContext
- **Tone:** Analytical, precise, data-focused

Narrative Profile (`src/domains/ai/profiles/narrative.ts`)

- **Purpose:** Story-based analysis of journal entries
- **Context:** Custom narrative context
- **Tone:** Storytelling, thematic analysis

Alert Profile (`src/domains/ai/profiles/alert.ts`)

- **Purpose:** Anomaly and alert generation
- **Response Format:** JSON array of alert items

Creating a New Profile

To add a new AI profile:

1. Create a new file in `src/domains/ai/profiles/` named `myprofile.ts`
2. Define the profile object with name, temperature, maxTokens, systemPrompt, and parseResponse
3. Add the profile to the profiles registry in `engine.svelte.ts`
4. Update the ProfileName type in `types.ts` to include your new profile
5. Create a corresponding context builder if needed (or reuse an existing one)

Example Template:

A new profile would follow this structure:

- Import the Profile type
- Create a system prompt that explains Claude's role
- Format data from the context object into readable text
- Define how Claude should format its response
- Export the profile object

Data Flow

Request to Response Flow

```
Component (Chat, Insight Widget, etc.)
↓
AIRequest {profile, prompt, contextHints?}
↓
engine.streamQuery() or engine.query()
↓
Look up Profile configuration
↓
Call context builder (e.g., buildChatContext)
↓
Fetch data sources:
- LedgerStore (logs)
- LocationStore (saved locations)
- TrackableStore (metric definitions)
- GoalStore (user goals)
↓
Transform raw data into structured context
- Tokenize notes
- Aggregate metrics
- Resolve locations
- Count interactions
↓
Build system prompt using context
↓
Send to Claude API
↓
Parse response using profile's parseResponse function
↓
Return to component
↓
Component displays or caches result
```

Location Resolution Flow

```
Log with coordinates (lat: 52.1181, lng: 4.4459)
↓
findNearestLocation() using LocationStore
↓
Check saved locations for nearest match (within 0.1 nautical miles)
↓
Found: "Thuis" (Home)
↓
Include in location context with:
- Name: "Thuis"
- Coordinates: lat/lng
- Visit count
- Last visit date
- Recent visit dates (last 7)
↓
```

Claude receives: "Thuis (lat: 52.1181, lng: 4.4459) – visited 367 times, recent visits: 2026-04-08, 2026-03-28, ..."

Data Enrichment Pipeline

Each data type goes through a consistent enrichment process:

1. **Extraction:** Query logs and extract relevant tokens/data
 2. **Filtering:** Remove nulls, empty values, irrelevant entries
 3. **Aggregation:** Group by key (metric, person, location, etc.)
 4. **Calculation:** Apply math rules, compute statistics
 5. **Ranking:** Sort by frequency, recency, or relevance
 6. **Truncation:** Keep top N items for performance
 7. **Formatting:** Convert to human-readable strings for Claude
-

Integration Guide for New Components

Adding AI to a New Feature

Step 1: Choose Your Context Type

Decide what data Claude needs:

- **RichContext:** Complex analysis needing full data (chat, insights, complex recommendations)
- **MetricsContext:** Goal and metric-focused (advice, goal tracking)
- **JournalContext:** Reflection and narrative (journaling, story generation)
- **HealthContext:** Health-specific analysis (health coaching, wellness)
- **LocationContext:** Spatial patterns (location-based recommendations)
- **CorrelationContext:** Finding relationships (pattern discovery)
- **TrendContext:** Momentum analysis (trend forecasting)
- **GoalProgressContext:** Goal achievement (progress tracking)
- **PatternContext:** Temporal patterns (schedule optimization, habit formation)

Step 2: Choose or Create a Profile

Determine if an existing profile matches your needs or create a new one.

Use existing if:

- Chat-like conversational analysis → use Chat profile
- Goal-focused advice → use Advice profile
- Reflection and journaling → use Journal profile

Create new if:

- Specialized role for Claude (e.g., "you are a nutritionist")
- Specific response format (JSON structure, specific fields)
- Different tone or focus area

Step 3: Create Your Component

In your component (e.g., a new dashboard widget or modal):

```
// Import the query function
import { query } from '../domains/ai/engine.svelte'
```

```
// In your component script:
let insightText = $state('')
let loading = $state(false)
let error = $state(null)

async function generateInsight() {
  loading = true
  error = null

  try {
    // Make the AI request
    const response = await query({
      profile: 'insight', // or your custom profile
      prompt: userPrompt, // the user's question or request
      contextHints: {
        dateRange: {
          from: startDate,
          to: endDate
        }
      }
    })

    insightText = response.content
  } catch (err) {
    error = err.message
  } finally {
    loading = false
  }
}
```

Step 4: Handle Real-time Responses (Optional)

For streaming/chat-like experiences:

```
import { streamQuery } from '../domains/ai/engine.svelte'

async function streamResponse(userMessage) {
  let fullResponse = ''

  try {
    await streamQuery(
      {
        profile: 'chat',
        prompt: userMessage,
        contextHints: { /* optional */ }
      },
      (chunk) => {
        // Called for each streamed text chunk
        fullResponse += chunk
        // Update UI reactively
        currentMessage = fullResponse
      }
    )
  }
```

```

    )
  } catch (err) {
    handleError(err)
  }
}

```

Step 5: Consider Caching

For expensive computations (insights, analysis):

```

// Store in widget.data for persistence
const cached = widget.data?.cachedInsight
const cacheDate = widget.data?.cachedDate

const today = dayjs().format('YYYY-MM-DD')
if (cached && cacheDate === today) {
  // Use cached result
  insightText = cached
} else {
  // Fetch fresh data
  const response = await query(/* ... */)
  // Cache the result
  widget.data.cachedInsight = response.content
  widget.data.cachedDate = today
  await saveWidget(widget)
}

```

Step 6: Control Context Hints

Pass context hints to customize the data provided:

```

// Limit to specific metrics
contextHints: {
  metrics: ['#sleep_aw', '#exercise', '#mood'],
  dateRange: { from: '2026-06-01', to: '2026-06-10' }
}

// Use custom date range
contextHints: {
  dateRange: {
    from: dayjs().subtract(7, 'days').format('YYYY-MM-DD'),
    to: dayjs().format('YYYY-MM-DD')
  }
}

// No hints: uses all data and default timeframe
// (e.g., last 30 days from settings)

```

Example: Building a Health Coach Widget

A practical example of adding AI to a new widget:

```

<script lang="ts">
  import { query } from '../../../domains/ai/engine.svelte'
  import { widget } from '../widget-class'

  let advice = $state('')
  let loading = $state(false)

  async function generateHealthAdvice() {
    loading = true

    try {
      const response = await query({
        profile: 'advice', // Goal-focused advice
        prompt: 'Based on my health metrics, what should I prioritize this week?',
        contextHints: {
          // Limit to health metrics
          metrics: ['#sleep_aw', '#exercise', '#heart_rate', '#steps_aw'],
          // Last 7 days for trend analysis
          dateRange: {
            from: dayjs().subtract(7, 'days').format('YYYY-MM-DD'),
            to: dayjs().format('YYYY-MM-DD')
          }
        }
      })

      advice = response.content
    } finally {
      loading = false
    }
  }
</script>

<div class="health-coach">
  <button on:click={generateHealthAdvice} disabled={loading}>
    {loading ? 'Generating...' : 'Get Health Advice'}
  </button>

  {#if advice}
    <div class="advice-box">{advice}</div>
  {/if}
</div>

```

Claude would receive HealthContext with only health-related metrics and generate targeted advice about sleep, exercise, and cardiovascular health.

Advanced Topics

Performance Optimization

Lazy Context Building

Contexts are built on-demand, not precomputed:

- Chat requests trigger `buildChatContext`
- Insight widgets trigger `buildChatContext`
- Goal trackers trigger `GoalProgressContext`

This minimizes API calls and ensures data freshness.

Date Range Management

The system automatically applies the "default timeframe" from user preferences when not specified. This:

- Ensures consistent data windows across features
- Allows users to control AI context window globally
- Reduces redundant data fetching

Default timeframe location: `src/domains/preferences/Preferences.ts` in the `AIConfig` type

Caching Strategy

Two-level caching pattern:

1. **Widget-level caching:** Store results in `widget.data` with cache date
2. **Session caching:** Keep recent responses in component state

For expensive operations (long-running analyses), check cache before fetching.

Extending Context Builders

To create a specialized context builder:

1. Define the return type interface in `types.ts`
2. Create the builder function following the pattern of existing builders
3. Include data fetching, transformation, and ranking
4. Export the function from `context-builder.ts`
5. Use in profiles via the Profile's context type selection

Error Handling

The system handles errors gracefully:

- **API Errors:** Caught in query/streamQuery, passed to component via error state
- **Data Fetch Errors:** Logged to console, return empty/undefined for that data type
- **Parsing Errors:** Profile `parseResponse` can throw; catch in component

Components should always handle the error state:

```
if (error) {  
  return `Unable to generate response: ${error}`  
}
```

Monitoring and Debugging

Key signals to monitor:

- **Loading state:** Indicates AI request in flight
- **Error messages:** Clarify what went wrong

- **Response latency:** API calls typically take 2-5 seconds
- **Context size:** Larger contexts take longer to process

For debugging:

1. Check the context object shape in component state
2. Verify profile configuration matches intended behavior
3. Validate context hints are correct (especially date ranges)
4. Check console for fetch errors or parsing failures

Summary: The Big Picture

The Nomie AI Integration Engine transforms raw tracking data into intelligent insights through a pipeline of:

1. **Data Fetching:** Queries logs, locations, goals, and interactions
2. **Context Building:** Organizes data into specialized formats for different use cases
3. **Profile Selection:** Routes to Claude with appropriate system prompts
4. **Response Processing:** Parses Claude's output for component use
5. **Caching:** Stores expensive computations for performance

New components can integrate by:

1. Choosing a context type
2. Selecting or creating a profile
3. Calling `query()` or `streamQuery()` with appropriate hints
4. Handling the response and error states
5. Optionally caching results

The architecture balances flexibility (many context types and profiles) with simplicity (consistent request/response patterns across all components).

File Reference Guide

File	Purpose
src/domains/ai/engine.svelte.ts	Core request handler and orchestrator
src/domains/ai/context-builder.ts	Context building functions for all data types
src/domains/ai/profiles/types.ts	TypeScript type definitions
src/domains/ai/profiles/chat.ts	Chat profile configuration
src/domains/ai/profiles/advice.ts	Advice/coaching profile
src/domains/ai/profiles/insight.ts	Insight generation profile
src/domains/ai/profiles/journal.ts	Journal reflection profile
src/domains/preferences/Preferences.ts	Settings including default timeframe
src/domains/locations/LocationStore.ts	Saved locations and resolution
src/domains/ledger/LedgerStore.ts	Log data source

<code>src/domains/goals/GoalStore.ts</code>	User goals
<code>src/domains/dashboard2/widget/types/*.svelte</code>	Example widget implementations

Conclusion

The AI Integration Engine provides a robust, extensible foundation for adding intelligence throughout Nomie. By understanding the context system, profiles, and integration patterns, developers can quickly add AI-powered features that understand the complete context of a user's tracking data.